SOFTWARE REQUIREMENTS SPECIFICATION

Grain Market Management Web Application

Submitted by

Muhammad Hasnat Ahmad

Roll No: S25BINFT7M04004

Submitted to

Dr. Umar Riaz

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Session 2025-26

Definitions and Abbreviations

Before reading this document, it is important to understand some basic terms and short forms that are used throughout. The table below explains all of them clearly.

Term / Short Form	What It Means
SRS	Software Requirements Specification — the name of this document
FYP	Final Year Project
Admin	The grain market owner who will use and manage this system
Client	Any farmer, buyer, or trader who is registered in the system
Transaction	A single record of money received or paid for a client
Balance Sheet	A summary that shows how much money came in, went out, and what is left
PDF	Portable Document Format — a file type used for printing and sharing reports
Django	A popular web development framework built with Python
ORM	A tool Django uses to talk to the database without writing raw SQL
FR	Functional Requirement — describes something the system must be able to do
UC	Use Case — describes how a user interacts with the system
ERD	Entity Relationship Diagram — shows how the database tables are connected
IUB	Islamia University of Bahawalpur
PostgreSQL	The database system used to store all data in this project
MVC	Model View Controller — the design pattern followed by Django

Summary

This document is the Software Requirements Specification for my Final Year Project. My project is called the Grain Market Management Web Application. I am Muhammad Hasnat Ahmad, a student of BS Information Technology at the Islamia University of Bahawalpur. This project is being done under the supervision of Dr. Umar Riaz.

The main idea behind this project is simple. Right now, grain market owners in Pakistan do everything by hand. They write client names in registers, calculate balances with a calculator, and send updates through phone calls. This takes a lot of time and mistakes happen very often. My system will solve this problem by moving everything to a website that is easy to use.

The system will be built using Python Django for the backend, HTML with Tailwind CSS and JavaScript for the front end, and PostgreSQL as the database. It will let the admin add clients, record daily transactions, see balance sheets, create PDF reports, and send messages to clients. This document explains everything the system will do and how it will work.

Table of Contents

Definitions and Abbreviations **1**
Summary **2**
Table of Contents **3**
1. Introduction **4**
1.1 Purpose 4
1.2 Scope 4
1.3 Product Perspective 5
1.4 User Characteristics 5
1.5 Similar Apps and Systems / Literature Review 6
1.6 Proposed Technologies 6
2. Requirements **7**
2.1 Functional Requirements 7
FR001 — Admin Login 7
FR002 — Add New Client 8
FR003 — Record Transaction 8
FR004 — Generate Balance Sheet 9
FR005 — Generate PDF Report 9
FR006 — Send Notification 10
FR007 — Daily Rate Management 10
2.2 Non-Functional Requirements 11
2.3 System Constraints 11
3. Use Cases and Flow of Processes **12**
3.1 Use Case 1 — Admin Login 12
3.2 Use Case 2 — Add New Client 14
3.3 Use Case 3 — Record Transaction 15
3.4 Use Case 4 — Generate PDF Report 17
4. System Data Model **19**
5. References **20**

1. Introduction

Grain markets are an important part of the economy in Pakistan, especially in cities like Bahawalpur. Every day, hundreds of farmers, buyers, and traders come to these markets to buy and sell grain. The people who run these markets have to keep track of many clients and a lot of money coming in and going out every single day.

The problem is that most grain market owners still use old methods. They write everything in notebooks or registers. If they make a calculation mistake, it can cause big problems. There is also no easy way to let clients know about their balance or send them updates. I decided to build a web application to solve all of these problems in one place.

This application will work on any computer or laptop with internet. The grain market owner just needs to open a browser, log in, and they can manage everything from one simple screen. No more paper registers, no more manual calculations, no more phone calls for updates.

1.1 Purpose

The purpose of writing this document is to clearly explain what my system will do. It is written for my supervisor Dr. Umar Riaz, the examination committee, and anyone else who needs to understand this project. This document describes every feature of the system, how users will interact with it, and what rules the system must follow.

1.2 Scope

My system will handle the following things:

- Adding new clients and giving each one a unique ID number
- Recording money received from a client or money paid to a client
- Automatically calculating and showing the current balance for each client
- Showing a balance sheet for any client with a date filter option
- Creating daily, weekly, and monthly reports and saving them as PDF files
- Sending email messages or notifications to one client or all clients at once
- Posting daily grain prices for different commodities on the dashboard
- Secure login for the admin so no one else can access the data

This system will not include online trading, buying or selling grain online, a mobile app, or support for more than one branch of the market. These things can be added in future versions if needed.

1.3 Product Perspective

This web application is a completely new system. It is not connected to any other software and does not need any other system to work. It will run on a cloud server and the admin can open it in any web browser like Chrome or Firefox. The data will be stored in a PostgreSQL database and the backend will be handled by the Django framework.

The system works in three layers. First is the browser where the admin sees everything and clicks buttons. Second is the Django backend which processes every request. Third is the PostgreSQL database which saves and retrieves all the data. The diagram below shows how these three layers connect to each other.

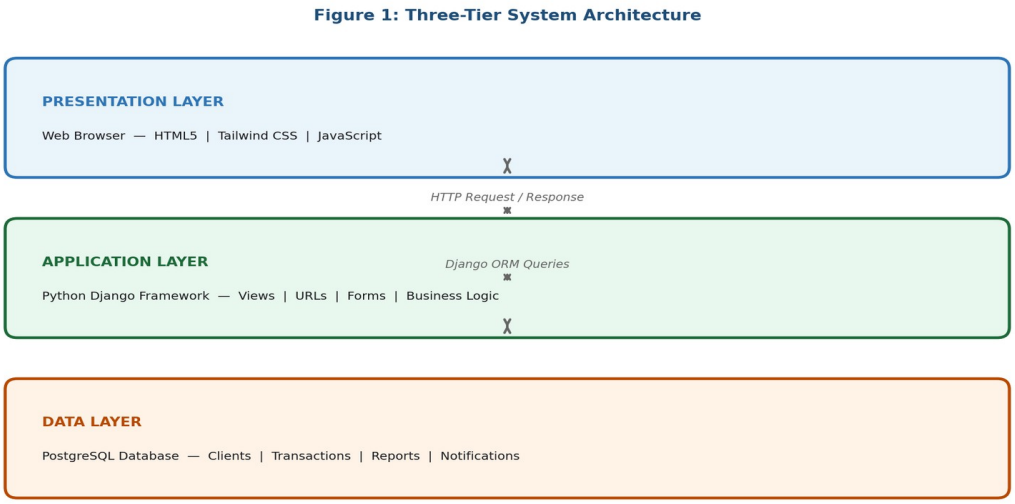


Figure 1: Three-Tier System Architecture

1.4 User Characteristics

There are three types of people who will be involved with this system. The table below explains who they are and what they can do.

User Role	What They Do	Technical Skill Needed
Administrator / Owner	This is the main person who uses the system. They add clients, record money, generate reports, and send messages to clients.	Basic computer skills are enough
Staff Member	A helper who assists the admin with entering data. This feature will be added in a future version of the system.	Basic computer skills are enough
Client (Passive)	The client does not log into the system. They only receive email notifications that the admin sends them.	No technical skill needed at all

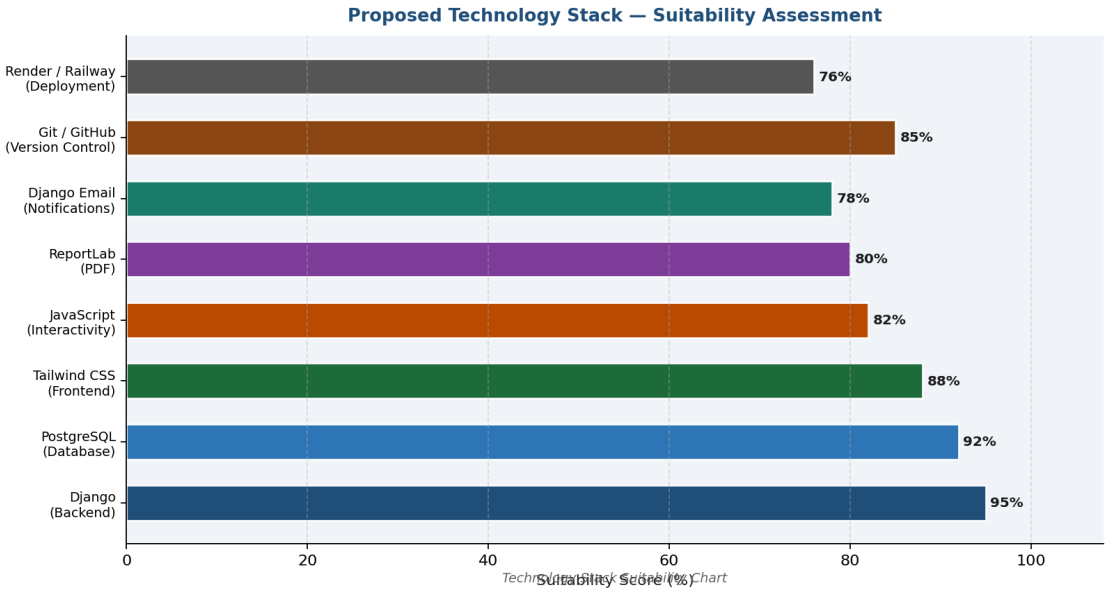
1.5 Similar Apps and Systems / Literature Review

Before building this system, I looked at other software tools that are already available for managing businesses. I found that none of them are suitable for a grain market in Pakistan. They are either too expensive, too complicated, or do not have the right features. The table below shows a comparison.

System Name	What It Can Do	Why It Does Not Work for Grain Markets
QuickBooks	Handles finances, invoices, and business reports	It is very expensive and too complex for a small grain market
Tally ERP 9	Manages accounts, stock, and balance sheets	It only works on desktop computers and needs a lot of training to use
Excel Spreadsheets	Manual data entry with formula calculations	Very easy to make mistakes and there is no notification feature
Custom PHP Scripts	Simple data forms used by some local shops	Poor design, no PDF export, and it breaks easily as the business grows
My Proposed System	Client management, transactions, reports, notifications, daily rates	Built specifically for grain market owners in Pakistan with all needed features

1.6 Proposed Technologies

I chose these technologies after doing research and getting approval from my supervisor. Each tool was selected because it is free, reliable, well documented, and suitable for a web-based system like this.



Technology Stack Suitability Assessment Chart

Component	Technology I Will Use	Why I Chose It
Backend	Python Django	It is easy to learn, has built-in login system, and works great with PostgreSQL
Frontend	HTML5, Tailwind CSS, JavaScript	Makes the interface clean, fast, and works on all screen sizes
Database	PostgreSQL	Free, powerful, and approved by my supervisor for this project
PDF Reports	ReportLab / WeasyPrint	Lets me create nicely formatted PDF files directly from Django
Email Sending	Django Email with SMTP	Simple to set up and works with Gmail to send notifications to clients
Code Structure	MVC Pattern	Keeps my code organized and easy to maintain or update later
Version Control	Git and GitHub	I can track all my changes and keep a backup of my code online
Hosting	Render or Railway	Free hosting platforms that support Django apps easily

2. Requirements

In this section I have described everything the system must be able to do. I divided the requirements into two parts. The first part is the functional requirements which are the actual features the system must have. The second part is the non-functional requirements which describe how well the system should perform. I also added a constraints section to mention the limitations of the current version.

2.1 Functional Requirements

Below are all the features my system will have. Each one is described in a table with its ID, purpose, who uses it, what input is needed, and what the system will give back as output.

FR001 — Admin Login

Field	Details
ID	FR001
Purpose	The admin needs to log in before using any part of the system. This keeps the data safe from unauthorized people.
User	Administrator
Input	Username and Password. The password must be at least 8 characters long and must have at least one number.
Output	If the login is correct, the admin is taken to the main dashboard. If wrong, an error message is shown.
Validation	After 5 wrong attempts, the account will be temporarily locked for security.

FR002 — Add New Client

Field	Details
ID	FR002
Purpose	The admin can register a new client in the system. Every new client gets a unique ID automatically.
User	Administrator
Input	Client full name, 11-digit phone number, home address, and opening balance (can be zero).
Output	The client is saved in the database and a unique client ID is generated and displayed on screen.
Validation	If the phone number already exists, the system will warn the admin. Empty required fields will show errors.

FR003 — Record Transaction

Field	Details
ID	FR003
Purpose	The admin records money coming in or going out for a specific client. The balance updates automatically.
User	Administrator
Input	Client ID, transaction type (Income or Expense), amount, a short description, and the date.
Output	The transaction is saved and the client balance is updated without any manual calculation.
Validation	The amount must be more than zero. If letters or symbols are entered, the system will reject it.

FR004 — Generate Balance Sheet

Field	Details
ID	FR004
Purpose	The admin can check the full financial summary of any client at any time.
User	Administrator
Input	Client ID. A start and end date can also be entered to filter the results.
Output	The screen shows total money received, total money paid, and the current balance for that client.
Validation	If there are no records for the selected time period, a clear message is shown.

FR005 — Generate PDF Report

Field	Details
ID	FR005
Purpose	The admin can generate a proper report and save or print it as a PDF file.
User	Administrator
Input	Report type (Daily, Weekly, or Monthly), date range, and an optional client filter.
Output	A properly formatted PDF report is ready to download immediately.
Validation	If no records match the selected filters, the system shows a No Records Found

	message.

FR006 — Send Notification

Field	Details
ID	FR006
Purpose	The admin can send a message or update to one specific client or to all clients at the same time.
User	Administrator
Input	Choose a recipient (one client or all clients) and type the message.
Output	The message is sent by email and a record of it is saved in the system with date and time.
Validation	The message box cannot be empty. Wrong email addresses will be rejected automatically.

FR007 — Daily Rate Management

Field	Details
ID	FR007
Purpose	The admin can post the latest grain prices every day. These prices will show on the dashboard.
User	Administrator
Input	Commodity name (like wheat, rice, or corn), price per unit, and the date.
Output	The rate is saved and displayed on the main dashboard for everyone to see.
Validation	The price must be a positive number. The same commodity cannot have two different prices on the same day.

2.2 Non-Functional Requirements

These requirements are about how the system should perform, not just what it should do. They are just as important as the features themselves.

Category	Requirement
Performance	Every page must open within 3 seconds on a normal internet connection. PDF reports must be ready within 10 seconds even when there is a lot of data.
Security	Passwords are never stored as plain text. Django hashes them automatically.

Category	Requirement
	All inputs are checked on the server to stop attacks. Sessions end after the admin is inactive for a while.
Usability	Any person with basic computer knowledge should be able to use this system without any training. The layout must look good and work correctly on screens that are 1024 pixels wide or more.
Reliability	The system should be available 99 percent of the time during working hours. No data should be lost during normal operations.
Maintainability	The code will be well commented and follow Django standards so that any developer can understand and update it easily. The database will follow 3NF normalization rules.
Scalability	The system must be able to handle at least 500 clients and 10,000 transactions without slowing down.
Compatibility	The system must work properly on Chrome, Firefox, Edge, and Safari browsers without any problems.

2.3 System Constraints

These are the limitations of the current version of the system. Some of them may be removed in future updates.

1. The system needs an internet connection to work. It cannot be used offline.
2. At the moment, the system only supports English. Urdu will be added in a future version.
3. This version is designed for one grain market only. Managing multiple branches is not included yet.
4. To generate PDF reports, the WeasyPrint or ReportLab library must be installed on the server.
5. Email notifications require a working Gmail or SMTP account to be configured on the server.
6. The screen must be at least 1024x768 pixels for the interface to display correctly.
7. All data operations require internet. There is no offline mode in this version.

3. Use Cases and Flow of Processes

A use case describes what happens when a user does something in the system. In my project, the main user is always the Admin. I have described four important use cases in detail below. Each one includes a table explaining the steps and a diagram showing the flow visually.

The diagram below shows all the things the admin can do inside the system at a high level.

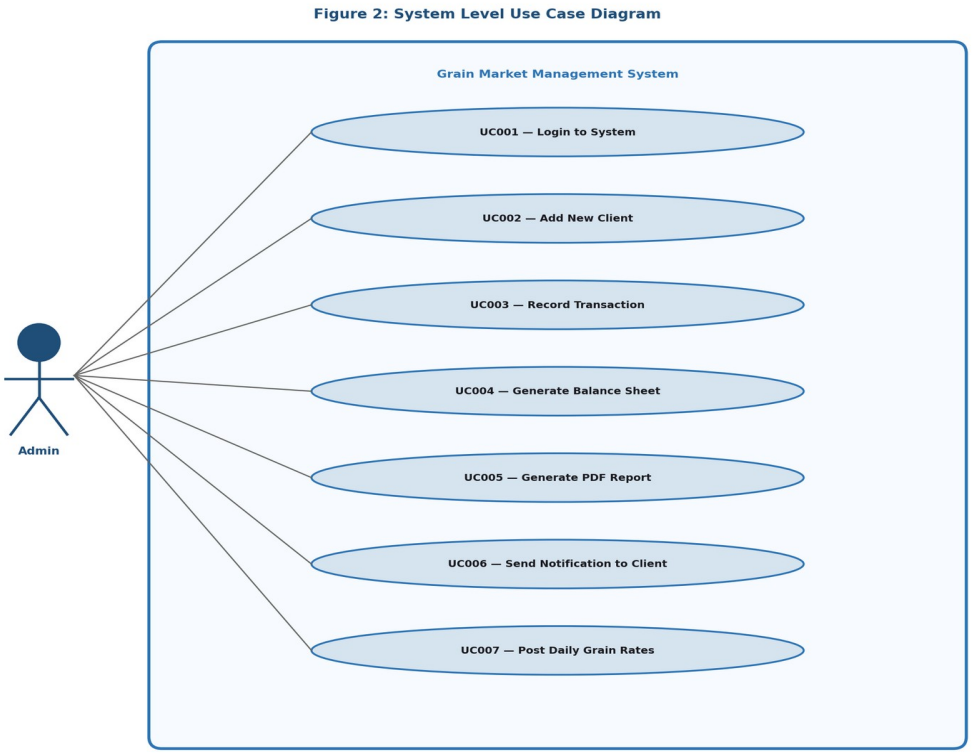


Figure 2: System Level Use Case Diagram

3.1 Use Case 1 — Admin Login

This use case describes how the admin logs into the system. It is the very first step before anything else can be done.

ID	UC001
Name	Admin Login
Description	The admin opens the login page, enters the username and password, and gets access to the system after the credentials are verified.
Requirement	FR001
Actor	Administrator

Precondition	The admin must already have an account created in the system.
Postcondition	The admin is logged in and can see the main dashboard.
Basic Flow	1. Admin opens the login page in the browser. 2. Admin types the username and password. 3. Admin clicks the Login button. 4. The system checks the entered details against the database. 5. If everything is correct, the system creates a session and takes the admin to the dashboard. Alternative Flow: If the username or password is wrong, the system shows an error and lets the admin try again. Exception: If the server is down or not reachable, a server error page is shown.

The diagram below shows the step by step flow of the login process:

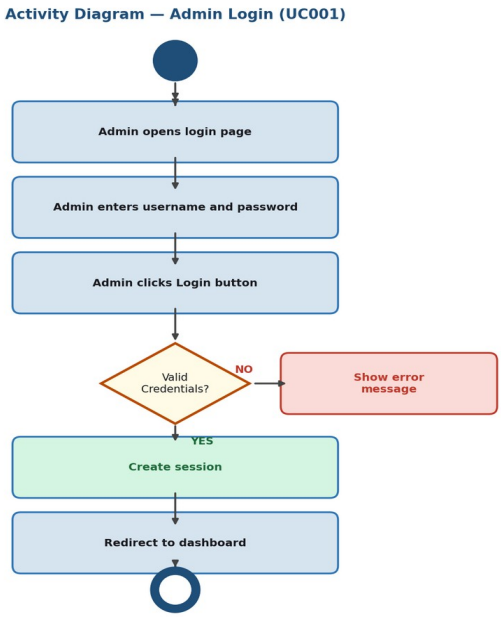


Figure 3: Activity Diagram — Admin Login

Figure 3: Activity Diagram — Admin Login

3.2 Use Case 2 — Add New Client

This use case describes how the admin registers a new grain market client. Once added, the client gets a unique ID that is used in all future transactions.

ID	UC002
Name	Add New Client
Description	The admin fills in the client details and submits the form. The system saves the client and generates a unique ID for them.
Requirement	FR002
Actor	Administrator
Precondition	The admin must be logged in. The same client should not already exist in the system.
Postcondition	The new client is saved in the database and a unique client ID is shown on screen.
Basic Flow	1. Admin goes to the Client Management section. 2. Admin clicks on the Add New Client button. 3. The system shows a form to fill in client details. 4. Admin enters the name, phone number, address, and opening balance. 5. Admin clicks Submit. 6. The system checks all the fields are filled in correctly. 7. The system saves the client and generates a unique client ID. 8. A success message is shown on screen. Alternative Flow: If the phone number already belongs to another client, a warning is shown before saving. Exception: If any required field is left empty, the form shows error messages next to those fields.

The diagram below shows the step by step flow of adding a new client:

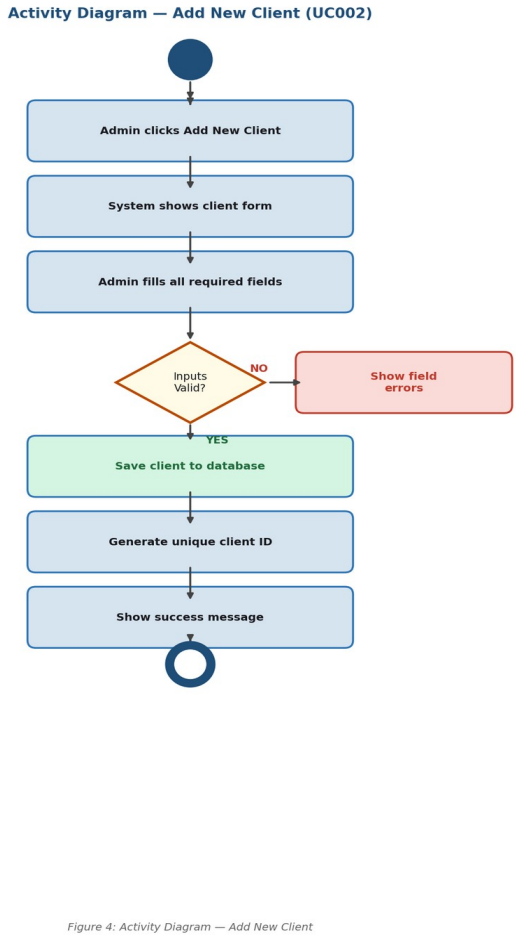


Figure 4: Activity Diagram — Add New Client

3.3 Use Case 3 — Record Transaction

This use case describes how the admin records a payment or receipt for a client. The system automatically updates the balance after each transaction so the admin does not need to calculate anything manually.

ID	UC003
Name	Record Transaction
Description	The admin selects a client, enters the transaction details, and submits. The system saves the record and updates the client balance.
Requirement	FR003
Actor	Administrator
Precondition	The admin must be logged in. The client must already be registered in the system.

Postcondition	The transaction is saved and the client balance is updated automatically.
Basic Flow	1. Admin goes to the Transactions section. 2. Admin selects the client using the name or unique ID. 3. Admin picks the transaction type — either Income or Expense. 4. Admin enters the amount, a short note about it, and the date. 5. Admin clicks Submit. 6. The system checks that the amount is a valid number greater than zero. 7. The transaction is saved and the client balance is updated. Alternative Flow: The admin can review and change the details before clicking Submit. Exception: If the amount is missing or contains letters, the system shows a validation error message.

The diagram below shows the step by step flow of recording a transaction:

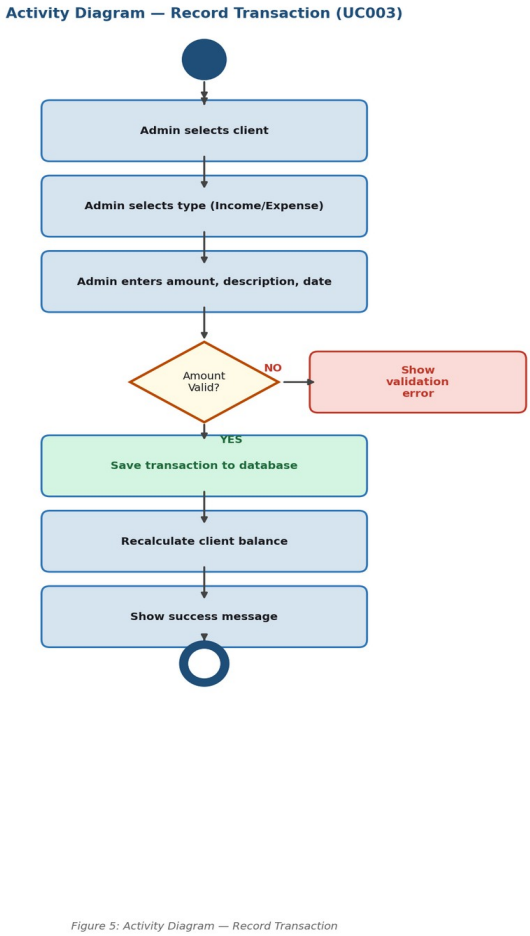


Figure 5: Activity Diagram — Record Transaction

3.4 Use Case 4 — Generate PDF Report

This use case describes how the admin creates a financial report and downloads it as a PDF file. The report can cover a single day, a week, or a full month.

ID	UC004
Name	Generate PDF Report
Description	The admin selects the report type and date range, then the system creates a PDF file that can be downloaded.
Requirement	FR005
Actor	Administrator
Precondition	The admin must be logged in. There must be transaction records available for the selected period.
Postcondition	A formatted PDF report is created and the admin can download it immediately.
Basic Flow	1. Admin goes to the Reports section. 2. Admin selects Daily, Weekly, or Monthly as the report type. 3. Admin sets a start date and end date. 4. Admin can also select a specific client or leave it as all clients. 5. Admin clicks Generate Report. 6. The system searches for matching records in the database. 7. The system puts together the data and creates a PDF file. 8. The PDF is ready and the admin can click to download it. Alternative Flow: The admin can preview the report on screen before downloading it. Exception: If no records are found for the selected dates, the system shows a No Records Found message.

The diagram below shows the step by step flow of generating a PDF report:

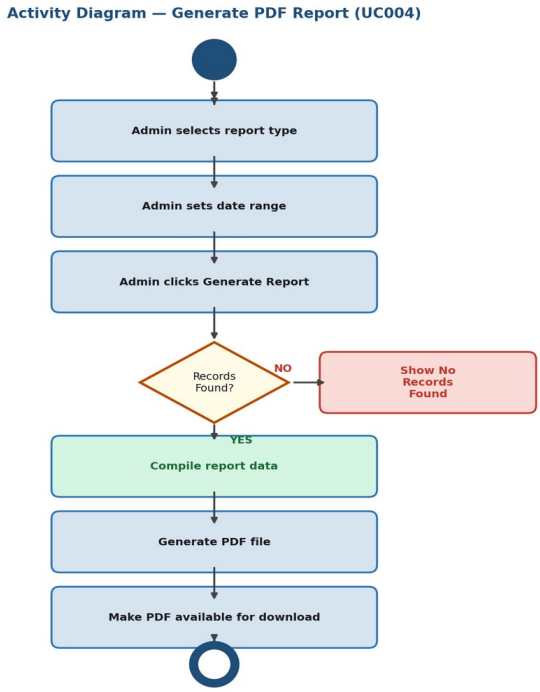


Figure 6: Activity Diagram — Generate PDF Report

Figure 6: Activity Diagram — Generate PDF Report

4. System Data Model

This section describes how the database of my system is organized. The database has six tables. Each table stores a different type of information. All these tables are stored in PostgreSQL and Django's ORM is used to create and manage them. I did not write any manual SQL. Django handles all of that automatically.

The diagram below shows all six tables and how they are connected to each other.

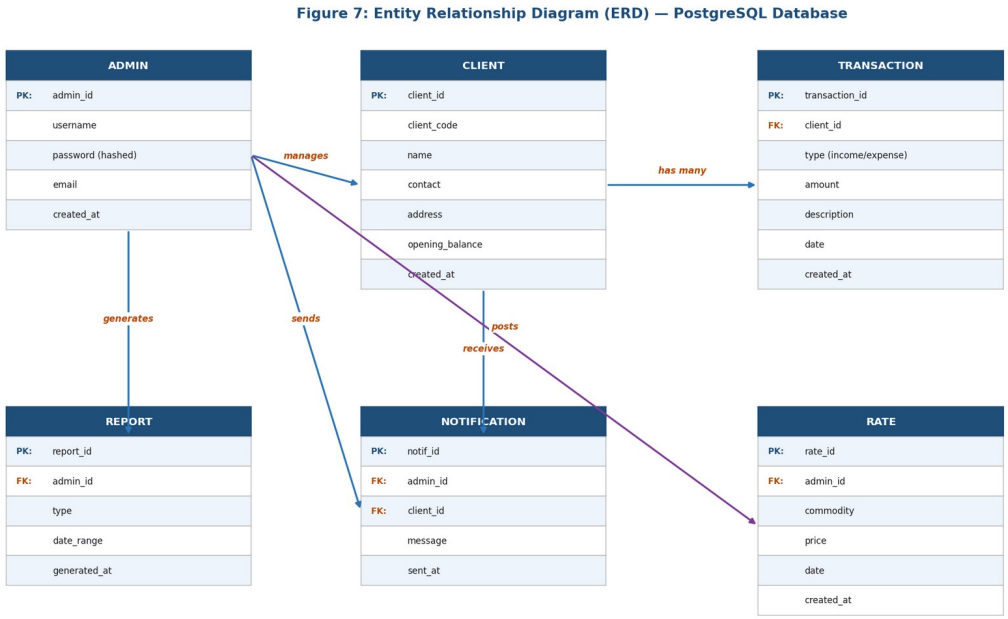


Figure 7: Entity Relationship Diagram (ERD) — PostgreSQL Database

Table Name	Primary Key	Important Columns	How It Connects
Admin	admin_id	username, password, email, created_at	Linked to all other tables because the admin manages everything
Client	client_id	client_code, name, contact, address, opening_balance	Connected to Transaction and Notification tables
Transaction	transaction_id	client_id (FK), type, amount, description, date	Each transaction belongs to one client
Report	report_id	admin_id (FK), type, date_range, generated_at	Each report is created by the admin

Table Name	Primary Key	Important Columns	How It Connects
Notification	notif_id	admin_id (FK), client_id (FK), message, sent_at	Sent by admin and linked to one or more clients
Rate	rate_id	admin_id (FK), commodity, price, date	Posted by admin and shown on the dashboard

5. References

- [1] M. H. Ahmad, "Grain Market Management Web Application — Final Year Project Proposal," The Islamia University of Bahawalpur, 2026.
- [2] IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Computer Society, 1998.
- [3] Django Software Foundation, "Django Documentation," [Online]. Available: https://docs.djangoproject.com. [Accessed: 2026].
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: https://www.postgresql.org/docs. [Accessed: 2026].
- [5] Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: https://tailwindcss.com/docs. [Accessed: 2026].
- [6] ReportLab Inc, "ReportLab PDF Library Documentation," [Online]. Available: https://www.reportlab.com/docs. [Accessed: 2026].
- [7] R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill Education, 2014.
- [8] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill Education, 2019.

— End of SRS Document —